

Laboratorio 2: Esquemas de detección y corrección de errores

Comparación de Hamming, CRC-32 y Fletcher sobre un canal con ruido configurable

1. Nombres y carnés

- Esteban Enrique Cárcamo Urizar — 23007
- Jorge Luis Felipe Aguilar Portillo — 23195

2. Descripción de la práctica

Se implementó una arquitectura de cinco capas que simula el intercambio de mensajes entre un cajero automático y el servidor de una entidad financiera sobre un canal no confiable. El emisor (cajero) está escrito en Python y el receptor (banco) en JavaScript sobre Node.js, comunicados por sockets TCP en el puerto 2705.

Cada mensaje recorre las capas de aplicación, presentación, enlace, ruido y transmisión. La capa de presentación codifica el texto a ASCII binario de 8 bits por carácter; la capa de enlace calcula la información de integridad y la concatena a la trama; la capa de ruido voltea cada bit con una probabilidad configurable solicitada al usuario, afectando por igual a los bits de datos y a los de redundancia. Del lado del receptor, la capa de enlace verifica la integridad y, cuando el algoritmo lo permite, corrige antes de entregar el mensaje a la capa de aplicación.

La capa de enlace no conoce ningún algoritmo en concreto: los solicita por nombre a un registro, de modo que incorporar un algoritmo nuevo requiere una sola línea. Se implementaron tres algoritmos, cada uno tanto en el emisor como en el receptor:

- Hamming genérico (n, m) , de corrección, con m configurable. Corrige un bit por bloque.
- CRC-32 (IEEE 802.3, polinomio reflejado 0xEDB88320), de detección.
- Fletcher, de detección, con palabras configurables de 8, 16 o 32 bits.

La implementación de CRC-32 se validó de forma independiente contra la función `zlib.crc32` de Python, reproduciendo el vector canónico de la cadena «123456789» con el valor 0xCBF43926. La equivalencia entre las versiones de Python y JavaScript de cada algoritmo se garantiza con un archivo de vectores de prueba compartido que ambas suites de pruebas cargan. En total se ejecutan 52 pruebas automatizadas (36 en Python y 16 en Node), todas en verde.

3. Resultados

El motor de simulación ejecutó 58,800 casos reproducibles con semilla determinista, combinando siete configuraciones de algoritmo, seis longitudes de mensaje (4 a 128 caracteres), siete probabilidades de error por bit (0 a 0.1) y 200 repeticiones por combinación.

3.1 Tasa de recuperación correcta del mensaje

Se considera éxito únicamente cuando el texto reconstruido por el receptor es idéntico al enviado. Los valores están en porcentaje.

Configuración	0	0.001	0.005	0.01	0.02	0.05	0.1
Hamming m=4	100.0	98.8	93.5	84.4	61.3	23.0	4.3
Hamming m=8	100.0	98.9	92.2	79.4	52.8	16.2	1.7
Hamming m=16	100.0	98.4	87.5	69.8	42.4	14.3	1.8
CRC-32 IEEE	100.0	73.0	35.7	20.3	7.3	0.5	0.0
Fletcher 8	100.0	74.0	37.9	22.9	9.7	1.2	0.0
Fletcher 16	100.0	73.0	35.7	20.3	7.3	0.5	0.0
Fletcher 32	100.0	70.6	31.0	15.3	4.3	0.1	0.0

Tabla 1. Tasa de éxito (%) según la probabilidad de error por bit.

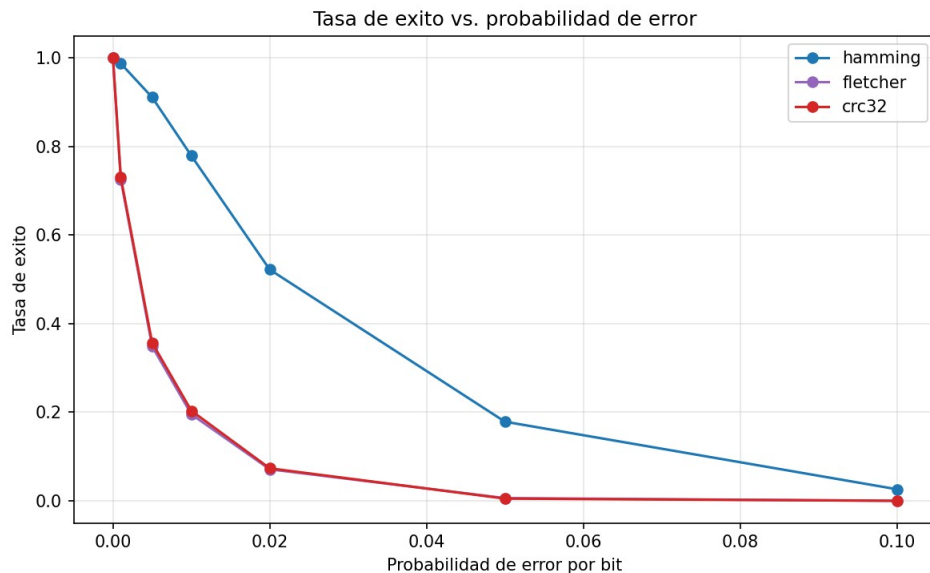


Figura 1. Tasa de éxito frente a la probabilidad de error por bit.

3.2 Overhead

El overhead de Hamming es constante porque la redundancia crece en proporción a los datos. En CRC-32 y Fletcher es un bloque fijo, de modo que su peso relativo se diluye conforme el mensaje crece. Los valores están en porcentaje sobre los bits de datos.

Configuración	4	8	16	32	64	128
Hamming m=4	75.0	75.0	75.0	75.0	75.0	75.0
Hamming m=8	50.0	50.0	50.0	50.0	50.0	50.0
Hamming m=16	31.2	31.2	31.2	31.2	31.2	31.2
CRC-32 IEEE	100.0	50.0	25.0	12.5	6.2	3.1
Fletcher 8	50.0	25.0	12.5	6.2	3.1	1.6
Fletcher 16	100.0	50.0	25.0	12.5	6.2	3.1
Fletcher 32	200.0	100.0	50.0	25.0	12.5	6.2

Tabla 2. Overhead (%) según la longitud del mensaje en caracteres.

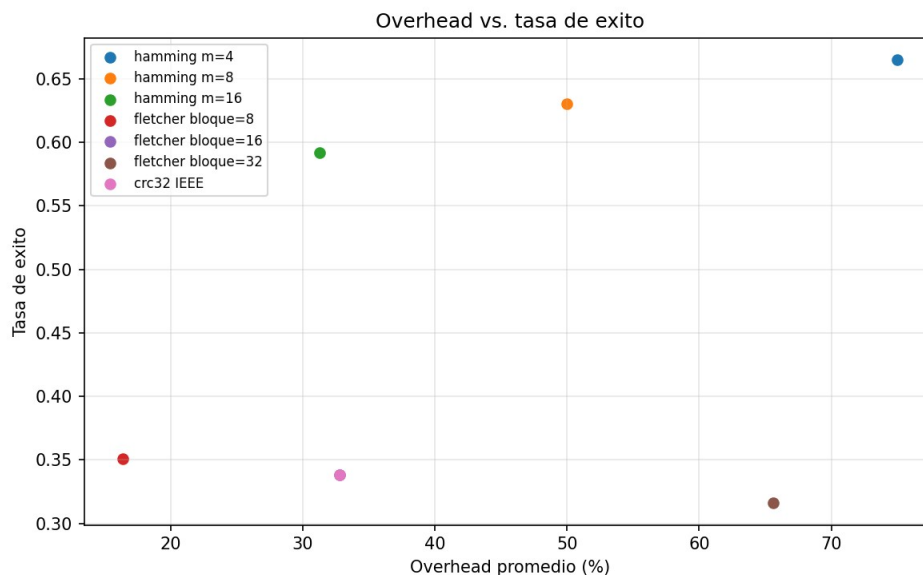


Figura 2. Relación entre overhead y tasa de éxito.

3.3 Falsos negativos

Un falso negativo ocurre cuando el algoritmo reporta que la trama está sana pero el mensaje entregado difiere del original. Es el modo de falla más peligroso, porque el error llega silenciosamente a la capa de aplicación.

Algoritmo	Casos	Falsos negativos
Hamming	25,200	9,347 (37.09 %)
Fletcher	25,200	1 (0.004 %)

Algoritmo	Casos	Falsos negativos
CRC-32	8,400	0 (0.00 %)

Tabla 3. Falsos negativos acumulados sobre todo el barrido.

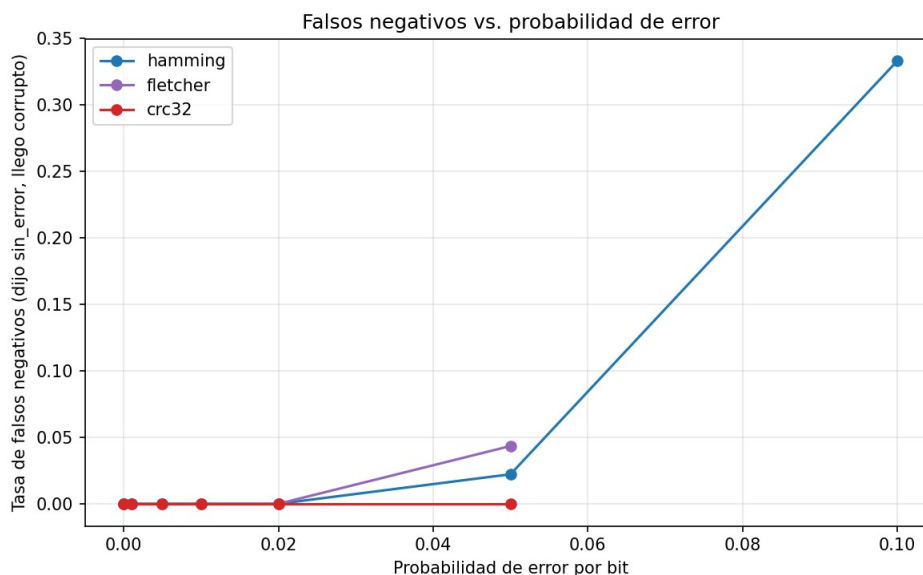


Figura 3. Falsos negativos frente a la probabilidad de error.

3.4 Evidencia de un falso negativo en la aplicación

Durante las pruebas de integración con Hamming y probabilidad 0.01, el receptor reportó haber corregido el error, pero el mensaje entregado a la capa de aplicación fue:

```
{"action": "withdraw", "date": {"amount": 25}}
```

La clave «data» se convirtió en «date». Un bloque llegó con dos bits alterados; el síndrome apuntó a una posición equivocada y Hamming introdujo un tercer error mientras creía estar corrigiendo. El banco no encontró el monto y respondió «Monto inválido». Es la limitación teórica del código manifestándose como un fallo de negocio silencioso.

4. Discusión

Hamming domina con claridad en la tasa de recuperación: con $m=4$ y una probabilidad de 0.01 recupera el 84.4 % de los mensajes, frente al 20.3 % de CRC-32. La comparación, sin embargo, es engañosa si se lee sola, porque mide cosas distintas. Hamming corrige y entrega el mensaje; CRC-32 y Fletcher solo detectan, y todo mensaje con al menos un bit alterado cuenta como fallo aunque el algoritmo haya hecho exactamente lo que debía. Para los algoritmos de detección, la tasa de éxito es en la práctica la probabilidad de que ningún bit se voltee, y por eso CRC-32 y Fletcher de 16 bits producen columnas idénticas: ambos añaden 32 bits y por tanto exponen la misma cantidad de bits al canal.

La tabla de falsos negativos invierte el veredicto. Hamming reportó éxito y entregó datos corruptos en el 37 % de los casos del barrido, y la proporción crece con la probabilidad de error

hasta superar el 98 %. La razón es estructural: al corregir siempre, Hamming nunca puede declarar que una trama es irrecuperable, así que cualquier bloque con dos o más bits alterados se convierte en corrupción silenciosa. CRC-32 no falló ni una vez en 8,400 casos, consistente con su probabilidad teórica de colisión de 2^{-32} . Fletcher falló una sola vez, con palabras de 8 bits, que es su configuración más débil.

Dentro de Hamming, reducir m mejora la corrección pero encarece el overhead: $m=4$ cuesta 75 % de redundancia y $m=16$ solo 31.2 %, con una diferencia de más de 14 puntos en tasa de éxito a probabilidad 0.01. El motivo es que con bloques pequeños hay menos posibilidades de que dos errores caigan en el mismo bloque, que es el escenario donde Hamming se rompe. En Fletcher ocurre lo contrario: palabras más grandes detectan mejor, pero el checksum ocupa el doble de bits y con mensajes cortos el overhead se vuelve prohibitivo, hasta 200 % para un mensaje de cuatro caracteres.

De ahí se desprende cuándo conviene cada enfoque. La corrección tiene sentido cuando retransmitir es caro o imposible —enlaces satelitales, medios de almacenamiento, transmisiones unidireccionales— y cuando la tasa de error es lo bastante baja como para que los errores dobles por bloque sean raros. La detección conviene cuando existe un canal de retorno para pedir retransmisión, que es el caso de este laboratorio: el cajero y el banco están conectados por TCP y pedir de nuevo el mensaje es trivial. En ese escenario, entregar un monto de retiro corrupto es mucho peor que pedir al usuario que repita la operación.

Ninguno de los tres algoritmos es superior en abstracto. La elección depende de si el costo dominante es el ancho de banda, la latencia de una retransmisión o la probabilidad de aceptar datos corruptos.

5. Conclusiones

- Hamming logró la mayor tasa de recuperación en todo el rango evaluado, con un máximo de 84.4 % a probabilidad 0.01 usando $m=4$, pero al costo de 75 % de overhead.
- Hamming es el más flexible ante tasas de error altas en cuanto a recuperación, y simultáneamente el menos confiable en cuanto a integridad: produjo falsos negativos en el 37.09 % de los casos.
- CRC-32 no registró un solo falso negativo en 8,400 casos, con un overhead que cae al 3.1 % en mensajes de 128 caracteres. Es la opción más sólida cuando existe canal de retorno.
- Reducir m en Hamming mejora la corrección porque disminuye la probabilidad de dos errores en un mismo bloque, a cambio de más redundancia.
- En Fletcher, palabras más grandes detectan mejor pero elevan el overhead hasta hacerlo inviable en mensajes cortos; con palabras de 8 bits apareció el único falso negativo del algoritmo.
- El aislamiento de la capa de enlace respecto a los algoritmos permitió incorporar un tercer algoritmo modificando una sola línea del registro, lo que confirma el valor de la arquitectura por capas.

6. Comentario grupal

El resultado que más nos sorprendió fue la magnitud de los falsos negativos de Hamming. Antes de correr las pruebas asumíamos que un algoritmo de corrección era estrictamente mejor que uno de detección, y la tabla de tasa de éxito parecía confirmarlo. Solo al medir cuántas veces Hamming entregaba datos corruptos diciendo que estaban bien entendimos que corregir y garantizar integridad son objetivos distintos, y que un algoritmo que siempre corrige nunca puede avisar que se rindió.

7. Citas y referencias

Hamming, R. W. (1950). Error detecting and error correcting codes. Bell System Technical Journal, 29(2), 147-160.

Fletcher, J. G. (1982). An arithmetic checksum for serial transmissions. IEEE Transactions on Communications, 30(1), 247-252.

IEEE. (2018). IEEE Standard for Ethernet, IEEE Std 802.3-2018, Clause 3.2.9: Frame Check Sequence (FCS) field.

Tanenbaum, A. S., & Wetherall, D. J. (2011). Computer Networks (5th ed.), Cap. 3: The Data Link Layer. Pearson.

Forouzan, B. A. (2013). Data Communications and Networking (5th ed.), Cap. 10: Error Detection and Correction. McGraw-Hill.